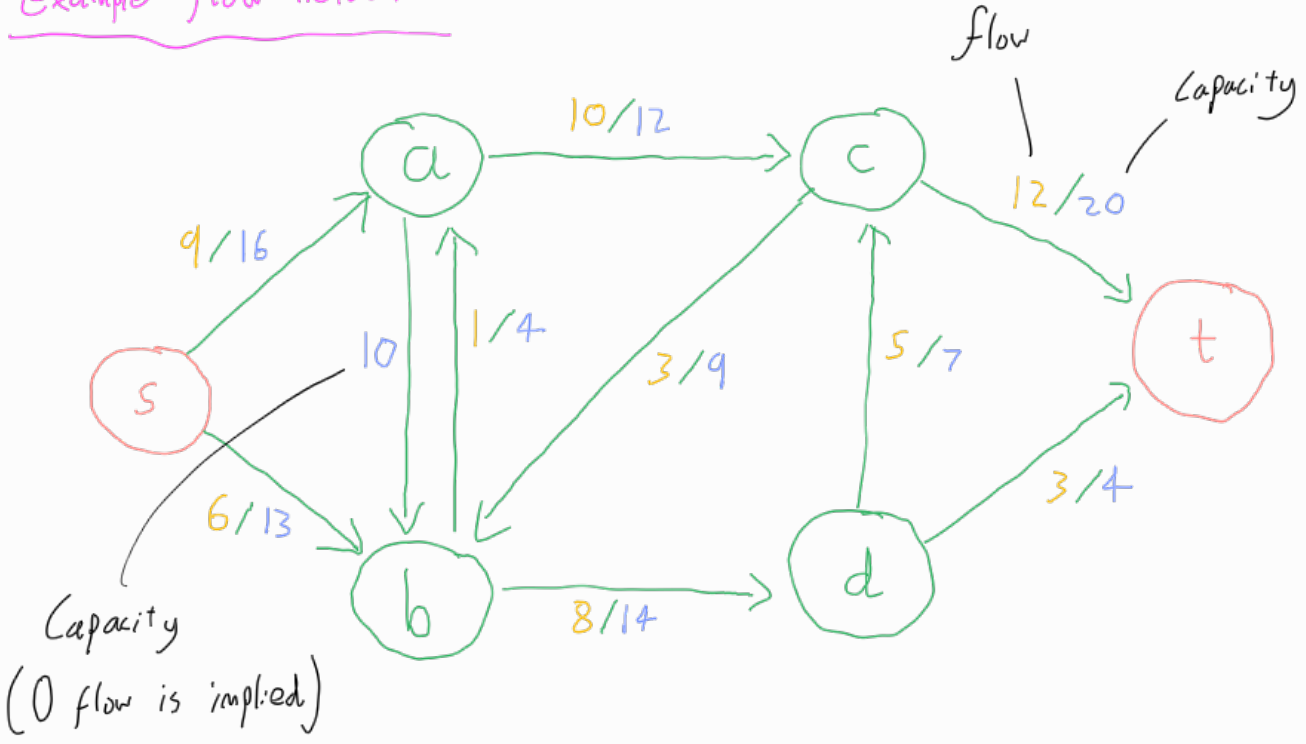
Flow Networks

A flow network is a graph $G = (V, E)$, with one vertex $s \in V$ designated the source and another vertex $t \in V$, $t \neq s$, the target. Every edge $(u, v) \in E$ has a non-negative capacity $c(u, v)$. If some edge is not in the graph, we assume its capacity is 0.

A flow $f(u, v)$ between two vertices u and v in a flow network is a real-valued function such that:

- for all $u, v \in V$, $f(u, v) \leq c(u, v)$ - Capacity Constraint
- for all $u, v \in V$, $f(u, v) = -f(v, u)$ - Skew-Symmetry
- for all $u \in (V - \{s, t\})$, $\sum_{v \in V} f(u, v) = 0$ - Flow Conservation
  - -> i.e. total flow out of a vertex = total flow into a vertex, except at s and t.

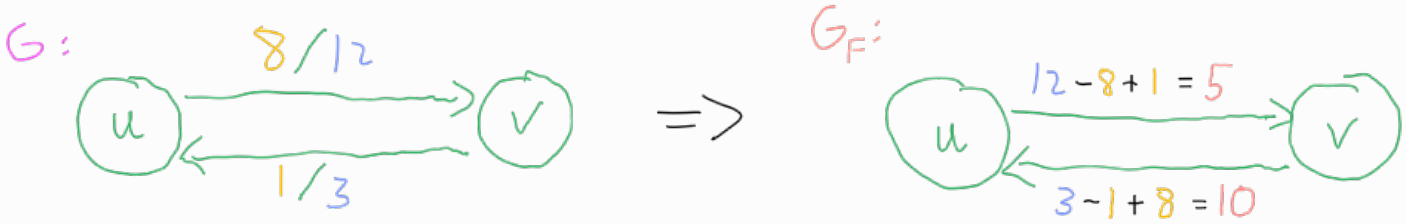
Example flow network



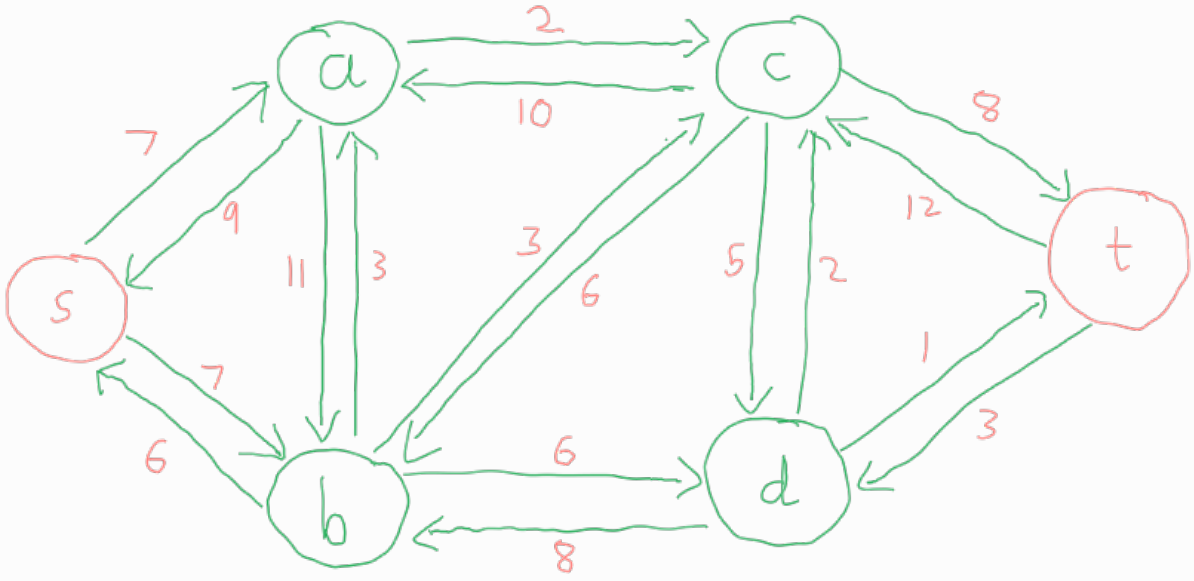
Given a flow network $G = (V, E)$ with a flow f, the residual network of G induced by f, denoted $G_f = (V, E_f)$, where the weight on each edge (u, v) is the residual capacity $c_f(u, v)$, given by:

$$c_f(u, v) = c(u, v) - f(u, v)$$

Note: in cases where there is negative flow across an edge (u,v), the residual capacity can exceed the capacity.



Residual network for our example



Given a flow network G and a flow f, an augmenting path P is a simple path in the residual network G_f from s to t.

The flow value along this path can be increased by $\min_{(u,v)\in P} c_f(u,v)$.

The Maximum Flow problem is the problem of finding the maximum possible total flow from s to t in a flow network G. The Ford-Fulkerson Algorithm provides a solution to this problem:

(Pseudocode)

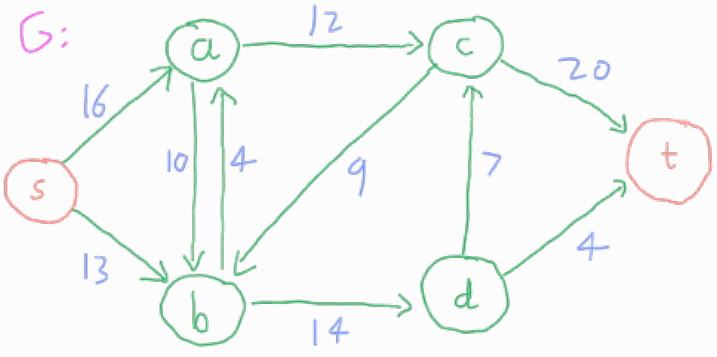
Input: G, s, t

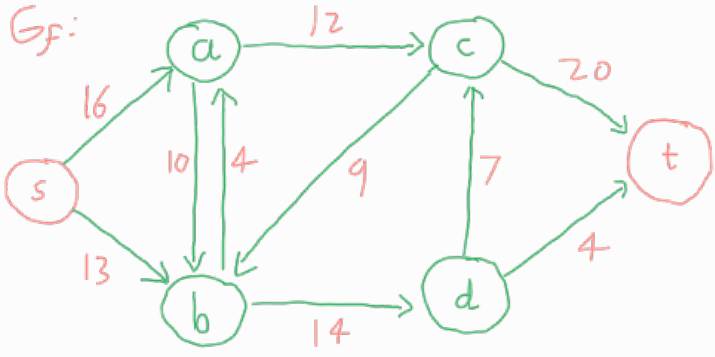
1: **for** each edge $(u, v) \in E$ **do**
2: $f(u, v) \leftarrow 0$
3: $f(v, u) \leftarrow 0$
4: **end for**
5: **while** there exists a path $P = s \rightsquigarrow t$ in the residual network G_f **do** {i.e. augmenting path in G}
6: $c_f(P) \leftarrow \min\{c_f(u, v) : (u, v) \in P\}$
7: **for** each edge $(u, v) \in P$ **do**
8: $f(u, v) \leftarrow f(u, v) + c_f(P)$
9: $f(v, u) \leftarrow -f(u, v)$
10: **end for**
11: **end while**

Note: the algorithm does not specify how we go about finding the augmenting paths. The implementation may make wrong or inefficient decisions as to which path to choose, but it is designed to be able to recover.

Example execution

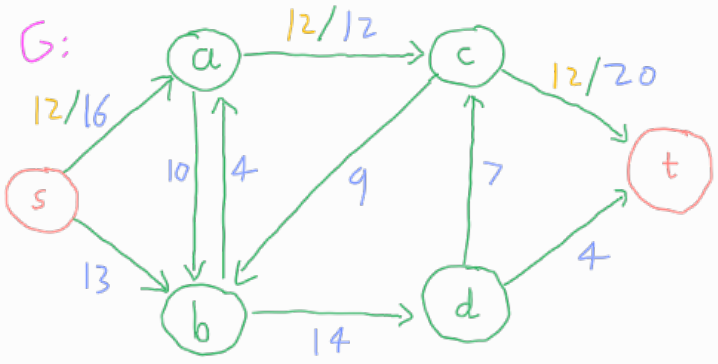
Initialise the flow to 0 across all edges

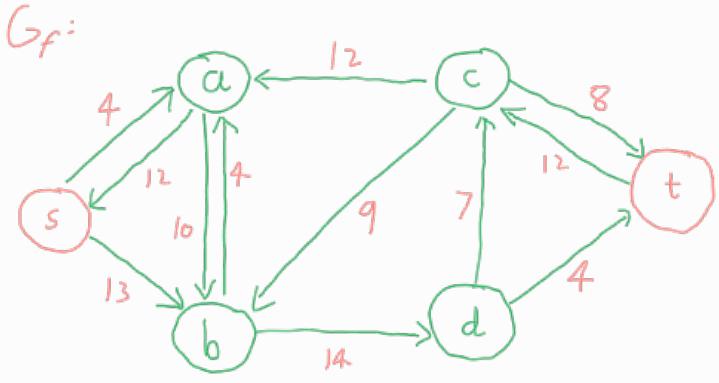




Choose an augmenting path. Since there is no specified way to do this, let's choose $P = s \to a \to c \to t$ for funsies.

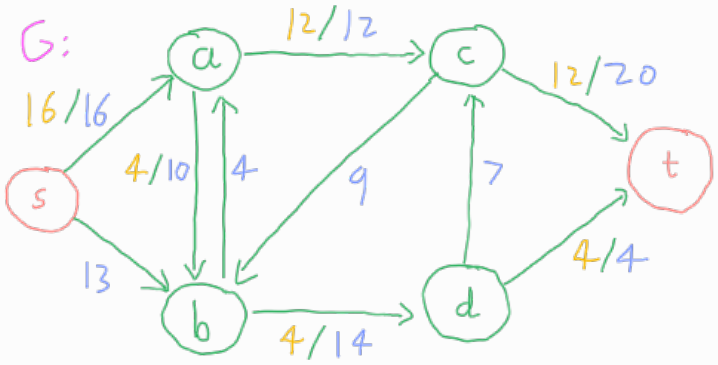
Add flow of $\min_{(u,v)\in P} c_f(u,v) = 12$ to each edge in P

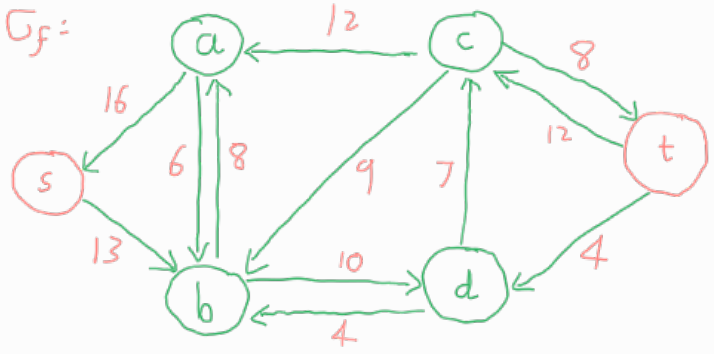




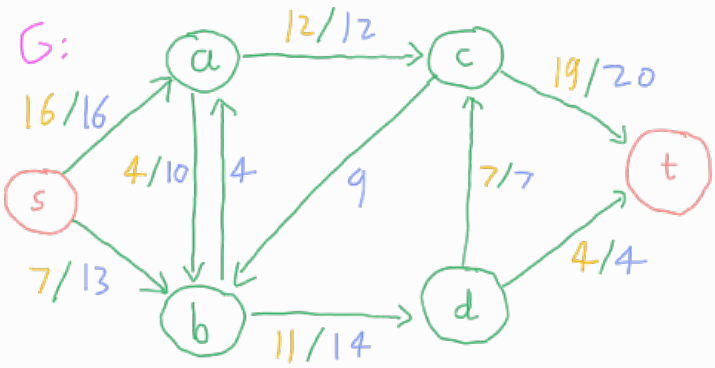
Now, let's choose $s \to a \to b \to d \to t$ for the memes

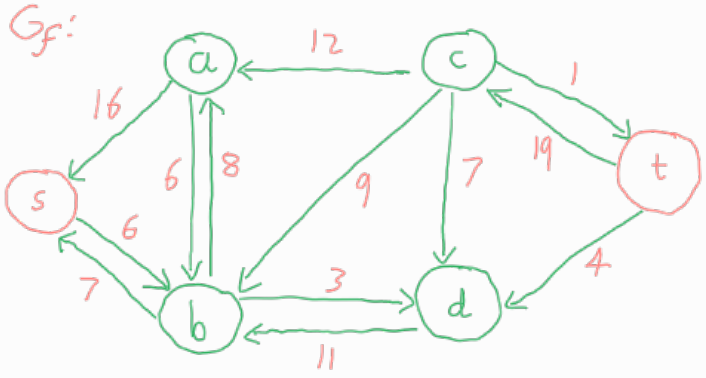
- Add flow of 4 along this path





The only remaining augmented path P is $s \to b \to d \to c \to t$. Add flow of 7 to this path





There are no more augmenting paths from s to t, therefore we have found our maximum flow: 23.

The running time of the algorithm depends on how we choose our augmenting paths. However, we know that every iteration of the while loop increases the value of the flow in the network by at least 1, therefore in the worst case this loop is executed $O(|f^*|)$ times, where $|f^*|$ is the maximum flow in the network. Each iteration takes $O(|E|)$ time, therefore the time complexity of the Ford-Fulkerson algorithm is $O(|f^*| \cdot |E|)$.

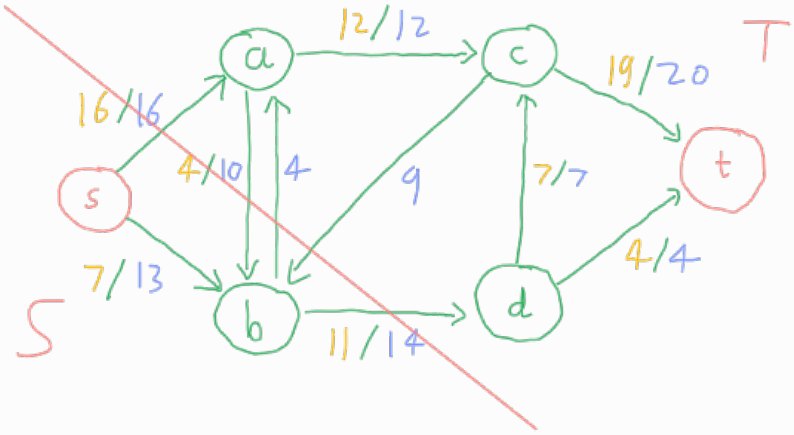
Note: an improved version of Ford-Fulkerson, the Edmonds-Karp Algorithm, has time complexity $O(|V| \cdot |E|^2)$

Sidequest: Cuts

A cut of a flow network $G = (V, E)$ is a partition of V into two sets S and T, containing s and t respectively.

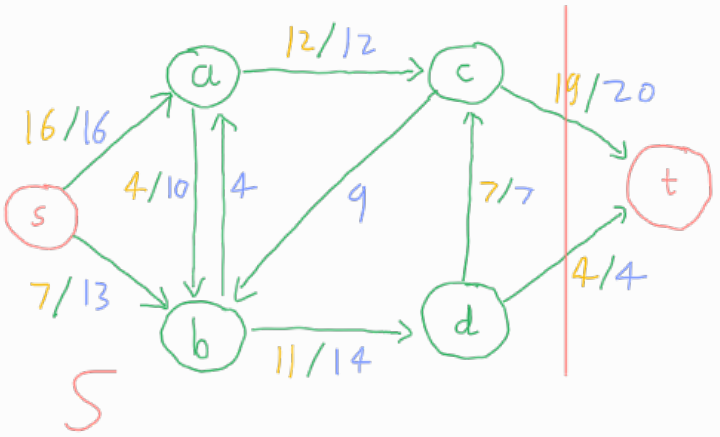
- $f(S, T)$ is the flow across the cut (S, T) – positive and negative edges
- $c(S, T)$ is the capacity of the cut (S, T) – only positive edges

Example Cuts



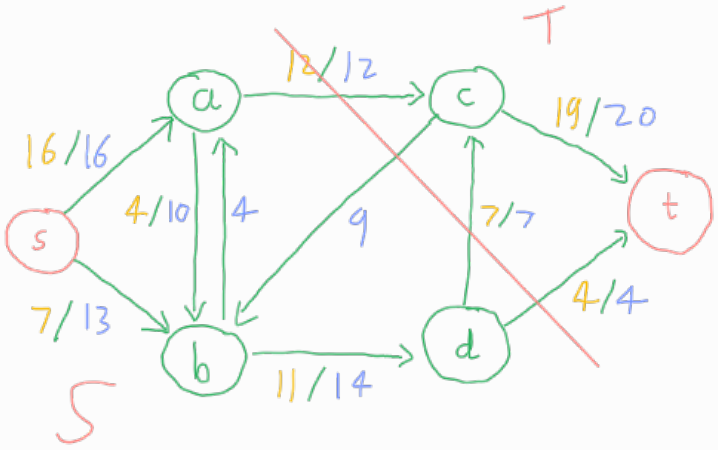
$f(S, T) = 16 + (-4) + 11 = 23$

$c(S, T) = 16 + 14 + 4 = 34$



$f(S,T) = 19+4 = 23$

$c(S,T) = 20+4 = 24$



$f(S,T) = 12+7+4 = 23$

$c(S,T) = 12+7+4 = 23$

↳ This is the Minimum Cut of our example graph.

The flow across any cut is the same – and is equal to the net flow from s to t in the network.

=> This makes sense: all units of flow must travel across any cut with one side containing the source and the other the target in order to get from source to target.

Therefore, the Minimum Cut determines the maximum flow from s to t in the network

=> The maximum flow in a network is equal to the capacity of the minimum cut

End of Sidequest

The Maximum Flow problem can be applied to the Maximum Bipartite Matching problem.

Given a bipartite graph G such that the vertices are in two sets X and Y such that all edges go from some vertex in X to some other vertex in Y, construct a flow network H with all vertices in G plus a source s and target t. Add edges from s to all vertices in X and from every vertex

in Y to t. Assign a capacity of 1 to every edge.

Apply Ford-Fulkerson to this graph.

Given the maximum flow determined by FF, let M be the set of edges e in the original graph G such that $f(e) = 1$ under this maximum flow.

M is a Maximum Matching of G.

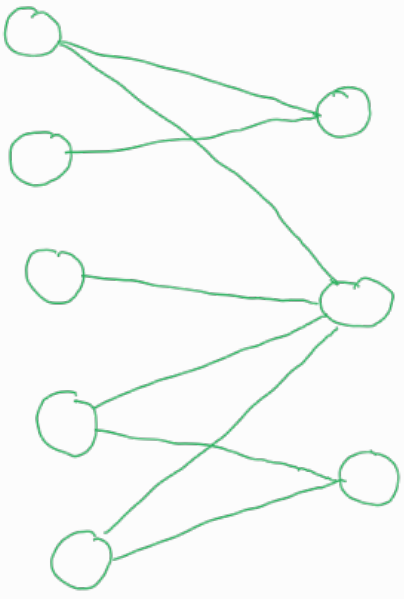
Conversely, given a Maximum Matching M in G, set $f(e) = 1$ for the edges in H that are in M. For each edge going into t / from s, if the other endpoint of this edge is one of the edges is in M, set the flow across this edge to 1 to satisfy the flow constraints.

The overall flow is a maximum flow in H

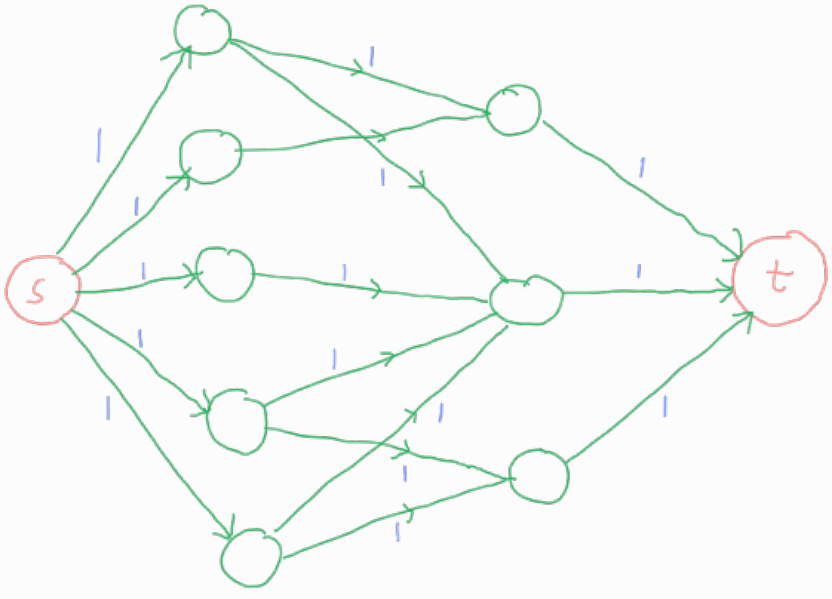
→ The maximum flow in H is equal to the number of edges in the Maximum Matching of G.

Example:

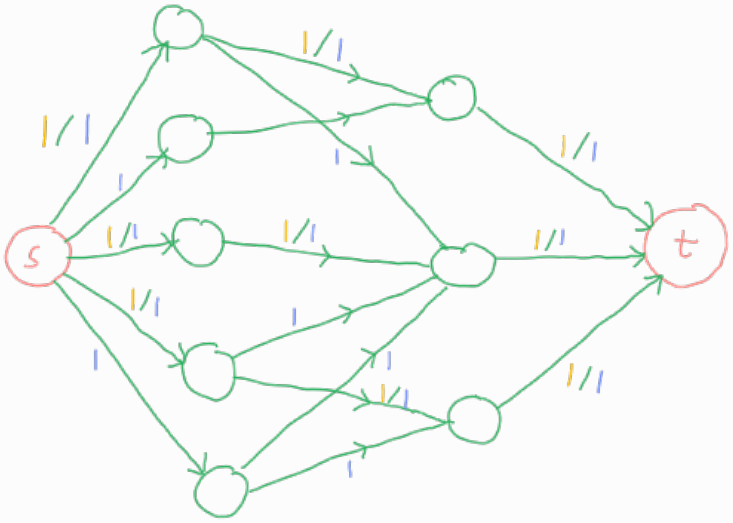
G =



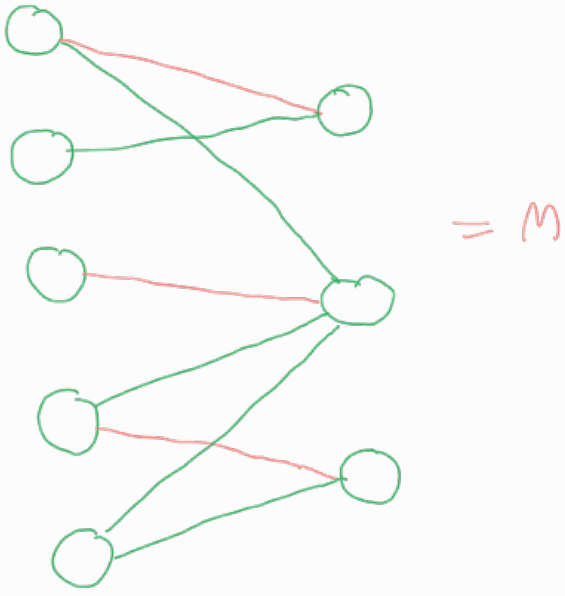
Construct H:



Apply FF:



$\Rightarrow$ Maximum matching is of size 3



$= M$

Note: there are multiple possible maximum matchings in this example. The exact matching depends on how we choose our augmenting path in FF.